

Part A (Analytical Exercises)

Q1:

- *Explain clearly why V_{π} is not useful in the MC development above?*
- **Answer:**

Chapter 4 proposed the policy iteration algorithm for control of the environment by the agent. This involved using the dynamics function $p(s', r|s, a)$ (which is assumed to be fully known by the agent) to evaluate a given policy, and then improving the policy by picking whichever action maximized the return (which was calculated using the dynamics function and the state value function $v_{\pi}(s')$).

If we are given a value function (for states), and knowledge of the dynamics function (the probabilities of reaching any state from the current one), then we have all the information we need to pick an optimal action for any state (without any need for the action value function). If we do not have knowledge of the dynamics function, then we have insufficient information to pick an optimal policy, and thus the state value function is useless. Since the Monte Carlo approach does assume any knowledge of $p(s', r|s, a)$, the value function V_{π} is not useful because we cannot use it to improve the policy.

Q2:

The MC algorithm so far (ref: p 99), requires an infinite number of episodes for Eval to converge on Q_{π_k} (step k). We can modify this algorithm to the practical variant where Eval is truncated (c.f., DynProg GPI). In this case:

2a:

- *Will we obtain Q_{π_k} from eval?*
- **Answer:** No, at any given timestep k , we do not necessarily obtain the converged Q_{π_k} using a truncated evaluation procedure.

2b:

- *If not why are we able to truncate Eval? Explain clearly.*
- **Answer:** Rather than fully completing evaluation before starting the improvement process, we use the two steps together (the GPI construct). So at each timestep, we move the action-value function closer towards Q_{π_k} iteratively (this holds given the assumption that all state-action pairs can be explored).

2c:

- *Assuming ES (i.e., thorough sampling of the $S \times A$ space), and the above truncated Eval_trunc, is it possible to converge on a sub-optimal policy π_c ? Is this a stable fixed point of the GPI for MC? Explain clearly.*
- **Answer:**

It is not possible to converge to a suboptimal policy, assuming ES and that the action-value function has converged to an optimal action-value function for that policy. We can verify this with a simple proof by contradiction (see appendix).

Stability can only be achieved when converging to both an optimal policy and optimal action-value function, however this convergence has not yet been formally proved for MC.

Q3:

- *Explain how you can synthesize a stochastic policy given what you know so far (you don't need to read ahead).*

- **Answer:**

Given state s and k actions, with probability β choose any action uniformly (probability of each action is $1/k$). With probability $1 - \beta$, choose $a_i^* = \operatorname{argmax}_i q_{\pi_k}(s, a_i)$

Part B (Practical Exercise)

Code the algorithm for MC Control (Off Policy) and apply this to the Cart Pole problem. You must discretize the environmental feedback (S) in order to solve this problem properly.

(Optional)

Explore the OpenAI gyms that are available, and find a discrete environment, and employ your MC control algorithm for it.

Answer the exercise questions above, and show logs from your code showing how long your solver can keep the pole balanced; the goal is clearly to balance it indefinitely (say for 10,000 steps). This should be submitted as a pdf: *assignment.pdf*. If you do optional, submit the logs as *optional.pdf*. For any logs, you must be very clear and explain what you are showing, and what it means. Just submitting a raw log with no comments or explanation is insufficient and can not be properly graded.

Introduction

An online resource was consulted for the purposes of rendering gymnasium environments in jupyter notebooks. This resource can be found at the following link and is referenced multiple times throughout this assignment:

- Source A: <https://community.latenode.com/t/how-to-render-gymnasium-environment-inside-jupyter-without-external-window/30582/4>

```
[1]: # ===== Taken from Source A =====  
from IPython import display  
import matplotlib.pyplot as plt  
%matplotlib inline  
# =====  
from abc import ABC, abstractmethod  
import gymnasium as gym  
import numpy as np  
import pprint # useful for printing nested items
```

We can copy over some of the basic code used in A1 and modify it for CartPole. Here is the code to set up the environment and a helper function for later:

```
[2]: ENV_ID, SEED = "CartPole-v1", 10  
NUM_TIMESTEPS_GOAL = 10000  
env = gym.make(ENV_ID, max_episode_steps=NUM_TIMESTEPS_GOAL)  
  
[3]: def _print_discrete_space(list_of_spaces):  
    names = ["Cart Position", "Cart Velocity", "Pole Angle", "Pole Angular Velocity"]  
    for i in range(len(list_of_spaces)):  
        print(f"{names[i]}: {list_of_spaces[i]}\n")
```

Approach

It is clear that the duration of this control algorithm depends directly on how we discretize the continuous feedback; so our execution loop will be a function of the fineness of discretization (most of the code for the execution loop is from Assignment 1).

The algorithm for off-policy MC control algorithm involves using an infinite amount of episodes but this is obviously infeasible. We need to generate enough episodes that the agent can actually solve/optimize the problem (the target policy must converge to a deterministic optimum). The instructions say the goal is for

the solver to balance it for 10,000 steps. Then our goal is to keep generating episodes until we finally reach an episode that is 10,000 timesteps long, however this is impractical so we implemented a maximum number of iterations of 100,000. Since the behaviour policy used to generate the episodes is arbitrarily soft and independent of the target policy being learned (except for coverage), we cannot use these episodes to test whether our algorithm has converged. So for every 1000 iterations, we generate an episode using the target policy as a test. Therefore, the agent has 100 “test attempts” to balance the cart for 10,000 timesteps, and 100,000 episodes to learn from. If the agent still does not accomplish this, it has failed.

We implemented three modified versions of the Off-Policy Control algorithm; each agent failed. We will present each implementation, discuss the changes, and afterward discuss the limitations of this approach. The “logs” for these algorithms can be found in Appendices B, C, and D.

Here is the main loop used to execute the environment (generate an episode) for agents 1 and 2:

```
[4]: def execute_environment(select_action, behaviour=None):
    """ Use <behaviour> policy if specified, otherwise use target policy """

    episode = []
    state, info = env.reset()

    reward, finished = None, False
    while not finished:
        action = select_action(state, behaviour)
        state, reward, terminated, truncated, info = env.step(action)
        episode.append((state, action, reward))
        finished = terminated or truncated
    env.close()
    return episode
```

Here is an abstract agent class that implements the policy but doesn't implement the discretization of the environment (these will be implemented by the child classes).

```
[5]: class AbstractAgent(ABC):

    def __init__(self, gamma):
        self.gamma = gamma
        self.MAX_ITERATIONS = 100000

    @abstractmethod
    def discretize_spaces(self):
        pass

    @abstractmethod
    def get_discrete(self):
        pass

    def select_action(self, state, b=None):
        s = self.get_discrete(state)
        if b is not None:
            return b(s)
        return self.pi[s]

    def greedy_update(self):
```

```

        self.pi = np.argmax(self.table[..., 0], axis=-1)

    def behavioral(self, state=None, action=None):
        if state is not None and action is not None:
            return 0.5
        return np.random.choice(2, 1)[0] # choose between action 0 and 1 with equal_
↪prob

    def control(self):

        self.table = np.random.random_sample(self.table_dims)
        self.table[..., 1] = 0
        self.greedy_update()

        output_logs = []
        success = False

        for num_iter in range(1, self.MAX_ITERATIONS+1):

            if not num_iter % 1000:
                ep = execute_environment(self.select_action)
                output_logs.append(f"Iteration {num_iter}, Test {num_iter // 1000}:_
↪target policy episode length {len(ep)}")
                if len(ep) == NUM_TIMESTEPS_GOAL:
                    success = True
                    break

            G,W = 0,1
            b = self.behavioral

            episode = execute_environment(self.select_action, b)
            for t in range(len(episode)-2,-1,-1):
                G = G*self.gamma + episode[t+1][-1]
                cont_state, A, _ = episode[t]
                S = self.get_discrete(cont_state)
                Q, C = self.table[S][A]
                C += W
                Q = Q + (W/C)*(G-Q)
                self.table[S][A] = [Q,C]

                optimal_action = np.argmax(self.table[S][...,0])
                self.pi[S] = optimal_action
                if optimal_action != A:
                    continue
                W *= 1/b(S,A)

        if success:
            print(f"Success! The agent was able to balance the pole for at least_
↪{NUM_TIMESTEPS_GOAL} timesteps.")
        else:
            print(f"Failure to meet goal after {self.MAX_ITERATIONS} iterations.")

```

```
return output_logs
```

Agent 1

Here is our first agent for Off-Policy MC agent implementation. It is a child of the Abstract class above and implements discretization. The observed feedback includes four values: cart position, cart velocity, angle (in radians), and angular velocity. Since it is infeasible to have an infinite state space for MC control, we manually limit the range of the cart velocity and the angular velocity. We chose the smallest range that included every single observed value from initial testing; these were $(-2.5, 2.5)$ and $(-3.5, 3.5)$ for cart velocity and angular velocity, respectively.

As mentioned earlier, discretization is necessary, and we decided to settle on 10 “bins” for each variable (so each variable has 10 possible values). Thus, the size of our state space is $|S| = 10^4 = 10000$, and since there are only two possible actions (push left or push right), our state-action is of size $|S \times A| = 10000 \times 2 = 20000$. For each state variables we ensure that there is an equal number of bins representing negative values as positive values (the edge case of 0 is irrelevant). Thus the number of bins are forced to be even.

The behaviour policy simply chooses between action 0 and 1 with equal probability for any state. Thus it is a soft policy and the assumption of coverage is still valid (any state-action pair possible under the target policy is possible under behaviour).

```
[6]: class Agent1(AbstractAgent):

    def __init__(self, gamma=0.9, num_bins=10, vel_range=(-2.5,2.5),
        angular_vel_range=(-3.5,3.5)):
        super().__init__(gamma)
        self.discretize_spaces(num_bins, vel_range, angular_vel_range)

    def _fix_num_bins(self, num_bins):
        n = int(num_bins)
        if n < 6:
            raise ValueError("Must specify number of bins of at least 2")
        return n+1 if n % 2 else n # return even number

    def _discretize(self, interval):
        l,u = interval
        mid = self.n // 2
        lower = np.linspace(l, 0, mid, endpoint=False)
        upper = np.linspace(0, u, mid+1)
        bins = np.concatenate([lower,upper])
        return bins

    def discretize_spaces(self, num_bins, vel_range, angular_vel_range):
        self.n = self._fix_num_bins(num_bins)
        self.table_dims = (self.n,self.n,self.n,self.n,2,2)
        pos_range = (-2.4, 2.4) # non terminal range for position
        angle_range = (-0.2095, 0.2095) # non terminal range for angle (radians)
        intervals = [pos_range, vel_range, angle_range, angular_vel_range]
        self.discrete_space = [self._discretize(x) for x in intervals]

    def get_discrete(self, s):
        def mapper(i):
```

```

        bin_index = np.digitize(s[i], self.discrete_space[i])-1
        return min(bin_index, self.n-1) # edge case for max val of interval
    return tuple(map(mappper, range(len(s))))

```

```

[7]: agent1 = Agent1()
      _print_discrete_space(agent1.discrete_space)
      agent1_output = agent1.control()

```

Cart Position: [-2.4 -1.92 -1.44 -0.96 -0.48 0. 0.48 0.96 1.44 1.92 2.4]

Cart Velocity: [-2.5 -2. -1.5 -1. -0.5 0. 0.5 1. 1.5 2. 2.5]

Pole Angle: [-0.2095 -0.1676 -0.1257 -0.0838 -0.0419 0. 0.0419 0.0838 0.1257
0.1676 0.2095]

Pole Angular Velocity: [-3.5 -2.8 -2.1 -1.4 -0.7 0. 0.7 1.4 2.1 2.8 3.5]

Failure to meet goal after 100000 iterations.

Agent 2

Agent 1 failed to converge to an optimal target policy. We realized that the range of starting values enforced by the environment is very small (from -0.05 to 0.05). This meant that for each of our variables, the starting value could only belong to one of the bins. While this is fine theoretically (we do not need the Exploring Starts assumption for Off-Policy MC), it means that very few of the states are being sampled frequently to make learning practical (theoretically we need to be able to guarantee each state-action pair is visited infinitely). Given our current discretization this would take an infeasibly long time unless we significantly increase the number of bins (which quickly becomes computationally intractable without distributed architecture).

We then remembered that there are only two possible actions: to push left or right. There is no way for the agent to decide how hard to push at any given timestep; it can only apply a pre-determined constant amount of pressure regardless of any state variables. The only thing the agent needs to decide is the direction to move the cart to. Therefore, the cart velocity and angular velocity are practically useless information! The only pertinent information for deciding on a direction to push are the position of the cart and angle of the pole (regardless of velocity).

We will now only consider cart position and pole angle and use 100 bins for each. Therefore the size of our state space $|S| = 100 \times 100 = 10000$ remains the same. By eliminating the useless velocity variables, we have gained a drastically finer discretization without an increase in the size of the state space!

Here is Agent 2:

```

[10]: class Agent2(AbstractAgent):

        def __init__(self, gamma=0.9, num_bins=100):
            super().__init__(gamma)
            self.discretize_spaces(num_bins)

        def _fix_num_bins(self, num_bins):
            n = int(num_bins)
            if n < 6:
                raise ValueError("Must specify number of bins of at least 6")
            return n+1 if n % 2 else n # return even number

```

```

def _discretize(self, interval):
    l,u = interval
    mid = self.n // 2
    lower = np.linspace(l, 0, mid, endpoint=False)
    upper = np.linspace(0, u, mid+1)
    bins = np.concatenate([lower,upper])
    return bins

def discretize_spaces(self, num_bins):
    self.n = self._fix_num_bins(num_bins)
    self.table_dims = (self.n,self.n,2,2)
    pos_range = (-2.4, 2.4) # non terminal range for position
    angle_range = (-0.2095, 0.2095) # non terminal range for angle (radians)
    intervals = [pos_range, angle_range]
    self.discrete_space = [self._discretize(x) for x in intervals]

def get_discrete(self, state):
    def mapper(i):
        bin_index = np.digitize(s[i], self.discrete_space[i])-1
        return min(bin_index, self.n-1) # edge case for max val of interval
    s = (state[0],state[2]) if len(state) > 2 else state
    return tuple(map(mapper, range(len(s))))

# Keep track of how long everything took and present this in appendix!

```

```

[11]: agent2 = Agent2()
       _print_discrete_space(agent2.discrete_space)
       agent2_output = agent2.control()

```

```

Cart Position: [-2.4   -2.352 -2.304 -2.256 -2.208 -2.16  -2.112 -2.064 -2.016 -1.968
-1.92  -1.872 -1.824 -1.776 -1.728 -1.68  -1.632 -1.584 -1.536 -1.488
-1.44  -1.392 -1.344 -1.296 -1.248 -1.2   -1.152 -1.104 -1.056 -1.008
-0.96  -0.912 -0.864 -0.816 -0.768 -0.72  -0.672 -0.624 -0.576 -0.528
-0.48  -0.432 -0.384 -0.336 -0.288 -0.24  -0.192 -0.144 -0.096 -0.048
0.      0.048 0.096 0.144 0.192 0.24   0.288 0.336 0.384 0.432
0.48   0.528 0.576 0.624 0.672 0.72   0.768 0.816 0.864 0.912
0.96   1.008 1.056 1.104 1.152 1.2    1.248 1.296 1.344 1.392
1.44   1.488 1.536 1.584 1.632 1.68   1.728 1.776 1.824 1.872
1.92   1.968 2.016 2.064 2.112 2.16   2.208 2.256 2.304 2.352
2.4   ]

```

```

Cart Velocity: [-0.2095 -0.20531 -0.20112 -0.19693 -0.19274 -0.18855 -0.18436 -0.18017
-0.17598 -0.17179 -0.1676  -0.16341 -0.15922 -0.15503 -0.15084 -0.14665
-0.14246 -0.13827 -0.13408 -0.12989 -0.1257  -0.12151 -0.11732 -0.11313
-0.10894 -0.10475 -0.10056 -0.09637 -0.09218 -0.08799 -0.0838  -0.07961
-0.07542 -0.07123 -0.06704 -0.06285 -0.05866 -0.05447 -0.05028 -0.04609
-0.0419  -0.03771 -0.03352 -0.02933 -0.02514 -0.02095 -0.01676 -0.01257
-0.00838 -0.00419 0.      0.00419 0.00838 0.01257 0.01676 0.02095
0.02514 0.02933 0.03352 0.03771 0.0419 0.04609 0.05028 0.05447
0.05866 0.06285 0.06704 0.07123 0.07542 0.07961 0.0838 0.08799
0.09218 0.09637 0.10056 0.10475 0.10894 0.11313 0.11732 0.12151

```



```

0.1257  0.12989  0.13408  0.13827  0.14246  0.14665  0.15084  0.15503
0.15922 0.16341  0.1676   0.17179  0.17598  0.18017  0.18436  0.18855
0.19274 0.19693  0.20112  0.20531  0.2095  ]

```

Failure to meet goal after 100000 iterations.

Agent 3

Agent 2 also failed to converge despite our significantly finer discretization. We came up with the hypothesis that there is too much noise in our behaviour policy; by choosing either action with equal probability this policy is ignoring new information about the state-action pairs which is used to update the target policy. So then our final idea is to bring the behaviour policy closer to the target policy whilst retaining its “softness”. This is done by providing a small epsilon term to the algorithm. At any point in an episode, the behaviour policy will choose the current best action (greedy argmax from target policy) with probability $1 - \epsilon$, and with probability ϵ it will pick the non-greedy action for exploration purposes. Thus our behaviour policy is now an ϵ -soft policy, but it still maintains coverage of the target policy.

To do this, however, we need to keep track of the probability for each action chosen by the behaviour policy. This was not needed before because both actions had an equal probability so we could simply use a hard code value of 0.5. Clearly, $\epsilon \neq 1 - \epsilon \neq 0.5$, so we make a slight modification to our environment execution code (note that we are not changing the environment, we are only changing the information the agent keeps track of when an episode is generated). We also decided to decrease our gamma value (thereby decreasing the long term return and ultimately punishing the agent for shorter episodes).

```

[12]: def execute_environment(select_action, b=None):
        """ Use <behaviour> policy if specified, otherwise use target policy """

        episode = []
        state, info = env.reset()

        reward, finished = None, False
        while not finished:
            if b is None:
                action = select_action(state)
                state, reward, terminated, truncated, info = env.step(action)
                episode.append((state, action, reward))
            else:
                action, prob = select_action(state, b)
                state, reward, terminated, truncated, info = env.step(action)
                episode.append((state, action, prob, reward))
            finished = terminated or truncated

        env.close()
        return episode

```

Though this agent is a child class of the previous agent, this change in the behaviour policy requires modifications in class methods which have already been implemented:

```

[13]: class Agent3(Agent2):

        def __init__(self, gamma=0.3, epsilon=0.1, num_bins=100):
            super().__init__(gamma, num_bins)
            self.epsilon = epsilon

```

```

def behavioral(self, state=None, action=None):
    s = self.get_discrete(state)
    greedy = self.pi[s]
    non_greedy = greedy ^ 1
    weights = (1-self.epsilon, self.epsilon)
    action = np.random.choice((greedy, non_greedy), p=weights)
    return (action, weights[int(action == non_greedy)])

def control(self):
    self.table = np.random.random_sample(self.table_dims)
    self.table[... , 1] = 0
    self.greedy_update()

    output_logs = []
    success = False
    for num_iter in range(1, self.MAX_ITERATIONS+1):

        if not num_iter % 1000:
            ep = execute_environment(self.select_action)
            output_logs.append(f"Iteration {num_iter}, Test {num_iter // 1000}:_
↳target policy episode length {len(ep)}")
            if len(ep) == NUM_TIMESTEPS_GOAL:
                success = True
                break

        G,W = 0,1
        b = self.behavioral
        episode = execute_environment(self.select_action, b)
        for t in range(len(episode)-2,-1,-1):
            G = G*self.gamma + episode[t+1][-1]
            cont_state, A, prob, _ = episode[t]
            S = self.get_discrete(cont_state)
            Q, C = self.table[S][A]
            C += W
            Q = Q + ((W/C)*(G-Q))
            self.table[S][A] = [Q,C]

            optimal_action = np.argmax(self.table[S][... ,0])
            self.pi[S] = optimal_action
            if optimal_action != A:
                continue
            W *= 1/prob

    if success:
        print(f"Success! The agent was able to balance the pole for at least_
↳{NUM_TIMESTEPS_GOAL} timesteps.")
    else:
        print(f"Failure to meet goal after {self.MAX_ITERATIONS} iterations.")

```

```
return output_logs
```

```
[14]: agent3 = Agent3()
      _print_discrete_space(agent3.discrete_space) # same as Agent 2
      agent3_output = agent3.control()
```

```
Cart Position: [-2.4   -2.352 -2.304 -2.256 -2.208 -2.16  -2.112 -2.064 -2.016 -1.968
-1.92  -1.872 -1.824 -1.776 -1.728 -1.68  -1.632 -1.584 -1.536 -1.488
-1.44  -1.392 -1.344 -1.296 -1.248 -1.2   -1.152 -1.104 -1.056 -1.008
-0.96  -0.912 -0.864 -0.816 -0.768 -0.72  -0.672 -0.624 -0.576 -0.528
-0.48  -0.432 -0.384 -0.336 -0.288 -0.24  -0.192 -0.144 -0.096 -0.048
0.      0.048  0.096  0.144  0.192  0.24   0.288  0.336  0.384  0.432
0.48   0.528  0.576  0.624  0.672  0.72   0.768  0.816  0.864  0.912
0.96   1.008  1.056  1.104  1.152  1.2     1.248  1.296  1.344  1.392
1.44   1.488  1.536  1.584  1.632  1.68    1.728  1.776  1.824  1.872
1.92   1.968  2.016  2.064  2.112  2.16    2.208  2.256  2.304  2.352
2.4   ]
```

```
Cart Velocity: [-0.2095 -0.20531 -0.20112 -0.19693 -0.19274 -0.18855 -0.18436 -0.18017
-0.17598 -0.17179 -0.1676  -0.16341 -0.15922 -0.15503 -0.15084 -0.14665
-0.14246 -0.13827 -0.13408 -0.12989 -0.1257  -0.12151 -0.11732 -0.11313
-0.10894 -0.10475 -0.10056 -0.09637 -0.09218 -0.08799 -0.0838  -0.07961
-0.07542 -0.07123 -0.06704 -0.06285 -0.05866 -0.05447 -0.05028 -0.04609
-0.0419  -0.03771 -0.03352 -0.02933 -0.02514 -0.02095 -0.01676 -0.01257
-0.00838 -0.00419 0.      0.00419  0.00838  0.01257  0.01676  0.02095
0.02514  0.02933  0.03352  0.03771  0.0419   0.04609  0.05028  0.05447
0.05866  0.06285  0.06704  0.07123  0.07542  0.07961  0.0838   0.08799
0.09218  0.09637  0.10056  0.10475  0.10894  0.11313  0.11732  0.12151
0.1257   0.12989  0.13408  0.13827  0.14246  0.14665  0.15084  0.15503
0.15922  0.16341  0.1676   0.17179  0.17598  0.18017  0.18436  0.18855
0.19274  0.19693  0.20112  0.20531  0.2095  ]
```

Failure to meet goal after 100000 iterations.

Limitations

So why did our algorithms fail to converge to a deterministic optimal policy? Perhaps an even finer discretization (larger state space) is needed. perhaps a more selective discretization that involves non-linear transformations of the current intervals are needed (to emphasize states that are less likely to be encountered).

It is also important to note that this is a very limited environment; as mentioned earlier, the only actions the agent can take is to decide the direction in which to push the cart. The agent cannot specify the amount of force to apply at any timestep, nor can it decide not to interfere. It seems nearly impossible that a constant force applied at each timestep could ever enable control of this problem. As the documentation notes, the “center of gravity of the pole varies the amount of energy needed to move the cart underneath it”. Perhaps this problem with the given state and action space could be better solved by non-tabular RL methods. Our conclusion is that for any commercial PC and GPU, solving this given problem (keeping the pole balanced indefinitely) with this algorithm and state-action space, is not possible.

Appendix A

Here is the full proof for the claim in Question 2c.

Claim: In Monte Carlo (with exploring starts), it is not possible to converge to a suboptimal policy π_c given that we have already converged to the optimal value function Q_{π_c}

Proof by contradiction:

Assume that we have converged to a suboptimal policy π_c (given we have converged to the optimal action-value function Q_{π_c}). Then there is at least one state $\hat{s}$ such that $\pi_c(\hat{s})$ yields a suboptimal action $\hat{a}$. Then, there exists some $a^* \neq \hat{a}$ such that $Q_{\pi_c}(\hat{s}, a^*) > Q_{\pi_c}(\hat{s}, \hat{a})$.

Recall that in each episode, at the first occurrence of any S_t, A_t pair, the return G is appended to the list $\text{Returns}(S_t, A_t)$. The table Q is then updated such that $Q(S_t, A_t)$ is updated with the average of $\text{Returns}(S_t, A_t)$. Finally, the policy $\pi(S_t)$ is updated action that maximizes the table Q at state S_t .

Since we have established that this algorithm has converged to π_c , then this means that either: - there is at least one remaining episode where some pairs involving $\hat{s}$ remain to be explored, but $\pi_c(\hat{s})$ will always update to $\arg\max_a Q_{\pi_c}(\hat{s}, a) = \hat{a}$ (Case 1). - or the policy update step no longer occurs because all state-action pairs for $\hat{s}$ have been explored in all episodes (we will call this Case 2)

- Case 1: We are in a particular episode where some actions remain to be evaluated for $\hat{s}$. We then have two further sub-cases:
 - Case 1A: The pair $(\hat{s}, a^*)$ has already been explored. But if this is true, then $Q_{\pi_c}(\hat{s}, a^*) > Q_{\pi_c}(\hat{s}, \hat{a}) \implies \arg\max_a Q_{\pi_c}(\hat{s}, a) = a^* \neq \hat{a}$. **Contradiction.**
 - Case 1B: The pair $(\hat{s}, a^*)$ will be explored at a future step in the current episode, but Case 1A showed us that when that happens $\pi_c(\hat{s})$ will be updated to a^* , which means the policy π_c has not converged yet. **Contradiction.**
 - Case 1C: The pair $(\hat{s}, a^*)$ was never explored and will not be explored. **Contradiction** (violates ES assumption).
- Case 2:
 - Using the same logic from Case 1, there must have been a point where $(\hat{s}, a^*)$ would have been explored and updated $\pi_c(\hat{s})$ to a^* . **Contradiction**

Our initial assumptions must inevitably lead to a contradiction, so we have proven that **it is not possible to converge to a suboptimal policy given we have converged to the optimal value function for that policy**.

Appendix B

Here are the logs from the Agent 1 target policy “tests”.

```
[15]: pprint.pprint(agent1_output)
```

```
['Iteration 1000, Test 1: target policy episode length 30',  
'Iteration 2000, Test 2: target policy episode length 13',  
'Iteration 3000, Test 3: target policy episode length 10',  
'Iteration 4000, Test 4: target policy episode length 11',  
'Iteration 5000, Test 5: target policy episode length 10',  
'Iteration 6000, Test 6: target policy episode length 24',  
'Iteration 7000, Test 7: target policy episode length 15',  
'Iteration 8000, Test 8: target policy episode length 12',  
'Iteration 9000, Test 9: target policy episode length 10',  
'Iteration 10000, Test 10: target policy episode length 10',  
'Iteration 11000, Test 11: target policy episode length 12',  
'Iteration 12000, Test 12: target policy episode length 21',  
'Iteration 13000, Test 13: target policy episode length 12',  
'Iteration 14000, Test 14: target policy episode length 9',  
'Iteration 15000, Test 15: target policy episode length 10',  
'Iteration 16000, Test 16: target policy episode length 20',  
'Iteration 17000, Test 17: target policy episode length 15',  
'Iteration 18000, Test 18: target policy episode length 12',  
'Iteration 19000, Test 19: target policy episode length 10',  
'Iteration 20000, Test 20: target policy episode length 10',  
'Iteration 21000, Test 21: target policy episode length 15',  
'Iteration 22000, Test 22: target policy episode length 10',  
'Iteration 23000, Test 23: target policy episode length 10',  
'Iteration 24000, Test 24: target policy episode length 9',  
'Iteration 25000, Test 25: target policy episode length 9',  
'Iteration 26000, Test 26: target policy episode length 11',  
'Iteration 27000, Test 27: target policy episode length 9',  
'Iteration 28000, Test 28: target policy episode length 28',  
'Iteration 29000, Test 29: target policy episode length 10',  
'Iteration 30000, Test 30: target policy episode length 10',  
'Iteration 31000, Test 31: target policy episode length 11',  
'Iteration 32000, Test 32: target policy episode length 22',  
'Iteration 33000, Test 33: target policy episode length 10',  
'Iteration 34000, Test 34: target policy episode length 59',  
'Iteration 35000, Test 35: target policy episode length 9',  
'Iteration 36000, Test 36: target policy episode length 10',  
'Iteration 37000, Test 37: target policy episode length 9',  
'Iteration 38000, Test 38: target policy episode length 59',  
'Iteration 39000, Test 39: target policy episode length 22',  
'Iteration 40000, Test 40: target policy episode length 20',  
'Iteration 41000, Test 41: target policy episode length 10',  
'Iteration 42000, Test 42: target policy episode length 11',  
'Iteration 43000, Test 43: target policy episode length 10',  
'Iteration 44000, Test 44: target policy episode length 10',  
'Iteration 45000, Test 45: target policy episode length 11',  
'Iteration 46000, Test 46: target policy episode length 9',  
'Iteration 47000, Test 47: target policy episode length 9',
```

'Iteration 48000, Test 48: target policy episode length 12',
'Iteration 49000, Test 49: target policy episode length 9',
'Iteration 50000, Test 50: target policy episode length 9',
'Iteration 51000, Test 51: target policy episode length 16',
'Iteration 52000, Test 52: target policy episode length 10',
'Iteration 53000, Test 53: target policy episode length 10',
'Iteration 54000, Test 54: target policy episode length 10',
'Iteration 55000, Test 55: target policy episode length 12',
'Iteration 56000, Test 56: target policy episode length 11',
'Iteration 57000, Test 57: target policy episode length 11',
'Iteration 58000, Test 58: target policy episode length 20',
'Iteration 59000, Test 59: target policy episode length 10',
'Iteration 60000, Test 60: target policy episode length 10',
'Iteration 61000, Test 61: target policy episode length 10',
'Iteration 62000, Test 62: target policy episode length 11',
'Iteration 63000, Test 63: target policy episode length 12',
'Iteration 64000, Test 64: target policy episode length 10',
'Iteration 65000, Test 65: target policy episode length 10',
'Iteration 66000, Test 66: target policy episode length 11',
'Iteration 67000, Test 67: target policy episode length 12',
'Iteration 68000, Test 68: target policy episode length 11',
'Iteration 69000, Test 69: target policy episode length 10',
'Iteration 70000, Test 70: target policy episode length 26',
'Iteration 71000, Test 71: target policy episode length 10',
'Iteration 72000, Test 72: target policy episode length 18',
'Iteration 73000, Test 73: target policy episode length 10',
'Iteration 74000, Test 74: target policy episode length 11',
'Iteration 75000, Test 75: target policy episode length 21',
'Iteration 76000, Test 76: target policy episode length 10',
'Iteration 77000, Test 77: target policy episode length 22',
'Iteration 78000, Test 78: target policy episode length 11',
'Iteration 79000, Test 79: target policy episode length 9',
'Iteration 80000, Test 80: target policy episode length 9',
'Iteration 81000, Test 81: target policy episode length 16',
'Iteration 82000, Test 82: target policy episode length 22',
'Iteration 83000, Test 83: target policy episode length 10',
'Iteration 84000, Test 84: target policy episode length 10',
'Iteration 85000, Test 85: target policy episode length 10',
'Iteration 86000, Test 86: target policy episode length 12',
'Iteration 87000, Test 87: target policy episode length 11',
'Iteration 88000, Test 88: target policy episode length 9',
'Iteration 89000, Test 89: target policy episode length 10',
'Iteration 90000, Test 90: target policy episode length 10',
'Iteration 91000, Test 91: target policy episode length 21',
'Iteration 92000, Test 92: target policy episode length 10',
'Iteration 93000, Test 93: target policy episode length 9',
'Iteration 94000, Test 94: target policy episode length 21',
'Iteration 95000, Test 95: target policy episode length 10',
'Iteration 96000, Test 96: target policy episode length 10',
'Iteration 97000, Test 97: target policy episode length 15',
'Iteration 98000, Test 98: target policy episode length 10',
'Iteration 99000, Test 99: target policy episode length 9',

'Iteration 100000, Test 100: target policy episode length 32']

Appendix C

Here are the logs from the Agent 2 target policy “tests”:

```
[16]: pprint.pprint(agent2_output)
```

```
['Iteration 1000, Test 1: target policy episode length 23',  
'Iteration 2000, Test 2: target policy episode length 62',  
'Iteration 3000, Test 3: target policy episode length 29',  
'Iteration 4000, Test 4: target policy episode length 11',  
'Iteration 5000, Test 5: target policy episode length 30',  
'Iteration 6000, Test 6: target policy episode length 33',  
'Iteration 7000, Test 7: target policy episode length 13',  
'Iteration 8000, Test 8: target policy episode length 15',  
'Iteration 9000, Test 9: target policy episode length 18',  
'Iteration 10000, Test 10: target policy episode length 71',  
'Iteration 11000, Test 11: target policy episode length 23',  
'Iteration 12000, Test 12: target policy episode length 44',  
'Iteration 13000, Test 13: target policy episode length 23',  
'Iteration 14000, Test 14: target policy episode length 13',  
'Iteration 15000, Test 15: target policy episode length 19',  
'Iteration 16000, Test 16: target policy episode length 74',  
'Iteration 17000, Test 17: target policy episode length 48',  
'Iteration 18000, Test 18: target policy episode length 38',  
'Iteration 19000, Test 19: target policy episode length 34',  
'Iteration 20000, Test 20: target policy episode length 36',  
'Iteration 21000, Test 21: target policy episode length 15',  
'Iteration 22000, Test 22: target policy episode length 34',  
'Iteration 23000, Test 23: target policy episode length 74',  
'Iteration 24000, Test 24: target policy episode length 33',  
'Iteration 25000, Test 25: target policy episode length 24',  
'Iteration 26000, Test 26: target policy episode length 49',  
'Iteration 27000, Test 27: target policy episode length 19',  
'Iteration 28000, Test 28: target policy episode length 51',  
'Iteration 29000, Test 29: target policy episode length 29',  
'Iteration 30000, Test 30: target policy episode length 10',  
'Iteration 31000, Test 31: target policy episode length 54',  
'Iteration 32000, Test 32: target policy episode length 32',  
'Iteration 33000, Test 33: target policy episode length 11',  
'Iteration 34000, Test 34: target policy episode length 28',  
'Iteration 35000, Test 35: target policy episode length 31',  
'Iteration 36000, Test 36: target policy episode length 36',  
'Iteration 37000, Test 37: target policy episode length 41',  
'Iteration 38000, Test 38: target policy episode length 58',  
'Iteration 39000, Test 39: target policy episode length 59',  
'Iteration 40000, Test 40: target policy episode length 11',  
'Iteration 41000, Test 41: target policy episode length 25',  
'Iteration 42000, Test 42: target policy episode length 28',  
'Iteration 43000, Test 43: target policy episode length 37',  
'Iteration 44000, Test 44: target policy episode length 29',  
'Iteration 45000, Test 45: target policy episode length 13',  
'Iteration 46000, Test 46: target policy episode length 92',  
'Iteration 47000, Test 47: target policy episode length 44',
```


'Iteration 48000, Test 48: target policy episode length 54',
'Iteration 49000, Test 49: target policy episode length 39',
'Iteration 50000, Test 50: target policy episode length 38',
'Iteration 51000, Test 51: target policy episode length 30',
'Iteration 52000, Test 52: target policy episode length 54',
'Iteration 53000, Test 53: target policy episode length 17',
'Iteration 54000, Test 54: target policy episode length 25',
'Iteration 55000, Test 55: target policy episode length 20',
'Iteration 56000, Test 56: target policy episode length 42',
'Iteration 57000, Test 57: target policy episode length 13',
'Iteration 58000, Test 58: target policy episode length 28',
'Iteration 59000, Test 59: target policy episode length 29',
'Iteration 60000, Test 60: target policy episode length 17',
'Iteration 61000, Test 61: target policy episode length 32',
'Iteration 62000, Test 62: target policy episode length 26',
'Iteration 63000, Test 63: target policy episode length 59',
'Iteration 64000, Test 64: target policy episode length 59',
'Iteration 65000, Test 65: target policy episode length 18',
'Iteration 66000, Test 66: target policy episode length 20',
'Iteration 67000, Test 67: target policy episode length 35',
'Iteration 68000, Test 68: target policy episode length 19',
'Iteration 69000, Test 69: target policy episode length 39',
'Iteration 70000, Test 70: target policy episode length 14',
'Iteration 71000, Test 71: target policy episode length 39',
'Iteration 72000, Test 72: target policy episode length 20',
'Iteration 73000, Test 73: target policy episode length 15',
'Iteration 74000, Test 74: target policy episode length 32',
'Iteration 75000, Test 75: target policy episode length 51',
'Iteration 76000, Test 76: target policy episode length 63',
'Iteration 77000, Test 77: target policy episode length 52',
'Iteration 78000, Test 78: target policy episode length 45',
'Iteration 79000, Test 79: target policy episode length 61',
'Iteration 80000, Test 80: target policy episode length 13',
'Iteration 81000, Test 81: target policy episode length 29',
'Iteration 82000, Test 82: target policy episode length 28',
'Iteration 83000, Test 83: target policy episode length 11',
'Iteration 84000, Test 84: target policy episode length 17',
'Iteration 85000, Test 85: target policy episode length 28',
'Iteration 86000, Test 86: target policy episode length 22',
'Iteration 87000, Test 87: target policy episode length 48',
'Iteration 88000, Test 88: target policy episode length 11',
'Iteration 89000, Test 89: target policy episode length 45',
'Iteration 90000, Test 90: target policy episode length 40',
'Iteration 91000, Test 91: target policy episode length 34',
'Iteration 92000, Test 92: target policy episode length 31',
'Iteration 93000, Test 93: target policy episode length 18',
'Iteration 94000, Test 94: target policy episode length 16',
'Iteration 95000, Test 95: target policy episode length 73',
'Iteration 96000, Test 96: target policy episode length 73',
'Iteration 97000, Test 97: target policy episode length 17',
'Iteration 98000, Test 98: target policy episode length 58',
'Iteration 99000, Test 99: target policy episode length 35',

'Iteration 100000, Test 100: target policy episode length 11']

Appendix D

Here are the logs from the Agent 3 target policy “tests”:

```
[17]: pprint.pprint(agent3_output)
```

```
['Iteration 1000, Test 1: target policy episode length 13',  
'Iteration 2000, Test 2: target policy episode length 11',  
'Iteration 3000, Test 3: target policy episode length 18',  
'Iteration 4000, Test 4: target policy episode length 20',  
'Iteration 5000, Test 5: target policy episode length 17',  
'Iteration 6000, Test 6: target policy episode length 10',  
'Iteration 7000, Test 7: target policy episode length 20',  
'Iteration 8000, Test 8: target policy episode length 17',  
'Iteration 9000, Test 9: target policy episode length 31',  
'Iteration 10000, Test 10: target policy episode length 12',  
'Iteration 11000, Test 11: target policy episode length 10',  
'Iteration 12000, Test 12: target policy episode length 15',  
'Iteration 13000, Test 13: target policy episode length 15',  
'Iteration 14000, Test 14: target policy episode length 12',  
'Iteration 15000, Test 15: target policy episode length 26',  
'Iteration 16000, Test 16: target policy episode length 12',  
'Iteration 17000, Test 17: target policy episode length 14',  
'Iteration 18000, Test 18: target policy episode length 10',  
'Iteration 19000, Test 19: target policy episode length 16',  
'Iteration 20000, Test 20: target policy episode length 12',  
'Iteration 21000, Test 21: target policy episode length 9',  
'Iteration 22000, Test 22: target policy episode length 10',  
'Iteration 23000, Test 23: target policy episode length 18',  
'Iteration 24000, Test 24: target policy episode length 12',  
'Iteration 25000, Test 25: target policy episode length 14',  
'Iteration 26000, Test 26: target policy episode length 10',  
'Iteration 27000, Test 27: target policy episode length 14',  
'Iteration 28000, Test 28: target policy episode length 20',  
'Iteration 29000, Test 29: target policy episode length 19',  
'Iteration 30000, Test 30: target policy episode length 12',  
'Iteration 31000, Test 31: target policy episode length 20',  
'Iteration 32000, Test 32: target policy episode length 9',  
'Iteration 33000, Test 33: target policy episode length 10',  
'Iteration 34000, Test 34: target policy episode length 60',  
'Iteration 35000, Test 35: target policy episode length 10',  
'Iteration 36000, Test 36: target policy episode length 10',  
'Iteration 37000, Test 37: target policy episode length 11',  
'Iteration 38000, Test 38: target policy episode length 11',  
'Iteration 39000, Test 39: target policy episode length 10',  
'Iteration 40000, Test 40: target policy episode length 9',  
'Iteration 41000, Test 41: target policy episode length 19',  
'Iteration 42000, Test 42: target policy episode length 11',  
'Iteration 43000, Test 43: target policy episode length 12',  
'Iteration 44000, Test 44: target policy episode length 11',  
'Iteration 45000, Test 45: target policy episode length 11',  
'Iteration 46000, Test 46: target policy episode length 9',  
'Iteration 47000, Test 47: target policy episode length 11',
```

'Iteration 48000, Test 48: target policy episode length 10',
'Iteration 49000, Test 49: target policy episode length 9',
'Iteration 50000, Test 50: target policy episode length 11',
'Iteration 51000, Test 51: target policy episode length 11',
'Iteration 52000, Test 52: target policy episode length 11',
'Iteration 53000, Test 53: target policy episode length 10',
'Iteration 54000, Test 54: target policy episode length 11',
'Iteration 55000, Test 55: target policy episode length 10',
'Iteration 56000, Test 56: target policy episode length 10',
'Iteration 57000, Test 57: target policy episode length 11',
'Iteration 58000, Test 58: target policy episode length 11',
'Iteration 59000, Test 59: target policy episode length 10',
'Iteration 60000, Test 60: target policy episode length 14',
'Iteration 61000, Test 61: target policy episode length 10',
'Iteration 62000, Test 62: target policy episode length 10',
'Iteration 63000, Test 63: target policy episode length 14',
'Iteration 64000, Test 64: target policy episode length 10',
'Iteration 65000, Test 65: target policy episode length 12',
'Iteration 66000, Test 66: target policy episode length 11',
'Iteration 67000, Test 67: target policy episode length 17',
'Iteration 68000, Test 68: target policy episode length 11',
'Iteration 69000, Test 69: target policy episode length 12',
'Iteration 70000, Test 70: target policy episode length 11',
'Iteration 71000, Test 71: target policy episode length 30',
'Iteration 72000, Test 72: target policy episode length 10',
'Iteration 73000, Test 73: target policy episode length 10',
'Iteration 74000, Test 74: target policy episode length 9',
'Iteration 75000, Test 75: target policy episode length 10',
'Iteration 76000, Test 76: target policy episode length 11',
'Iteration 77000, Test 77: target policy episode length 13',
'Iteration 78000, Test 78: target policy episode length 11',
'Iteration 79000, Test 79: target policy episode length 9',
'Iteration 80000, Test 80: target policy episode length 14',
'Iteration 81000, Test 81: target policy episode length 10',
'Iteration 82000, Test 82: target policy episode length 13',
'Iteration 83000, Test 83: target policy episode length 13',
'Iteration 84000, Test 84: target policy episode length 9',
'Iteration 85000, Test 85: target policy episode length 9',
'Iteration 86000, Test 86: target policy episode length 24',
'Iteration 87000, Test 87: target policy episode length 17',
'Iteration 88000, Test 88: target policy episode length 13',
'Iteration 89000, Test 89: target policy episode length 12',
'Iteration 90000, Test 90: target policy episode length 15',
'Iteration 91000, Test 91: target policy episode length 9',
'Iteration 92000, Test 92: target policy episode length 54',
'Iteration 93000, Test 93: target policy episode length 11',
'Iteration 94000, Test 94: target policy episode length 47',
'Iteration 95000, Test 95: target policy episode length 19',
'Iteration 96000, Test 96: target policy episode length 24',
'Iteration 97000, Test 97: target policy episode length 17',
'Iteration 98000, Test 98: target policy episode length 19',
'Iteration 99000, Test 99: target policy episode length 28',

'Iteration 100000, Test 100: target policy episode length 11']